

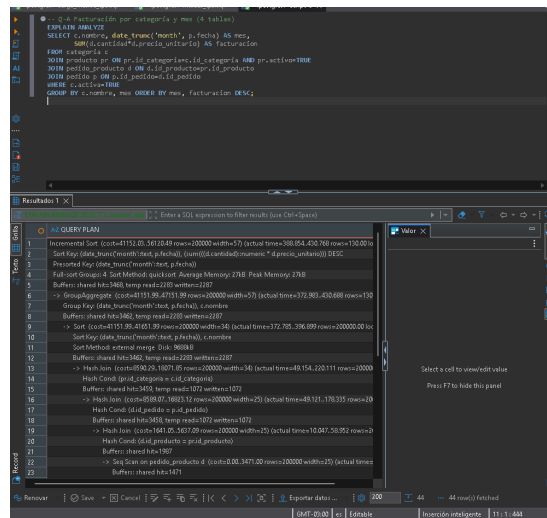
TP4 Semana 4 — Reportes analíticos, joins y ventana

1. Las dos analíticas que elegí y por qué tardan

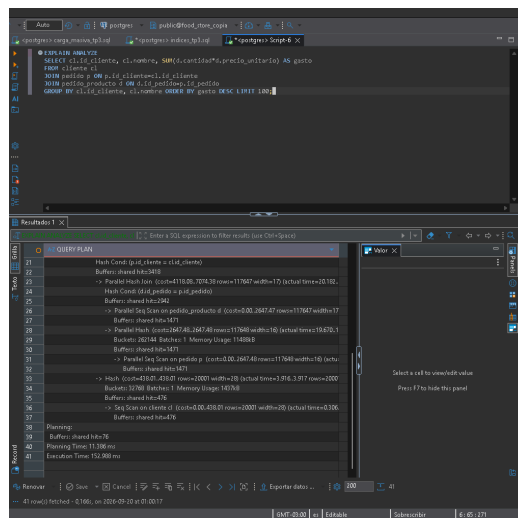
A diferencia del TP3, donde era un filtro en una tabla, acá el costo está en juntar 3-4 tablas y agregar 200k filas. Elegí dos reales: **Q-A facturación por categoría y mes** (categoria-producto-pedido_producto-pedido, 4 tablas, GROUP por mes) y **Q-B ranking de gasto por cliente Top 100** (cliente-pedido-detalle, 3 tablas). Las corrí con EXPLAIN ANALYZE antes de tocar nada.

Q-A me dio alrededor de **430 ms para 130 filas**. El plan tiene **3 Hash Join en cadena** y un **Sort que se desborda a disco (external merge, 9688 kB, temp read/write 2283)**. Empieza con Seq Scan en pedido_producto (200k) porque tiene que leer todo para agrupar. Q-B me dio **152.988 ms con 2 Parallel Hash Join** más Parallel Seq Scan en pedido y detalle. Tiene sentido: para agregar todo, hashear es lo más barato, la hash de pedido entra en 11 MB y la de clientes en 1.4 MB.

Q-A antes:



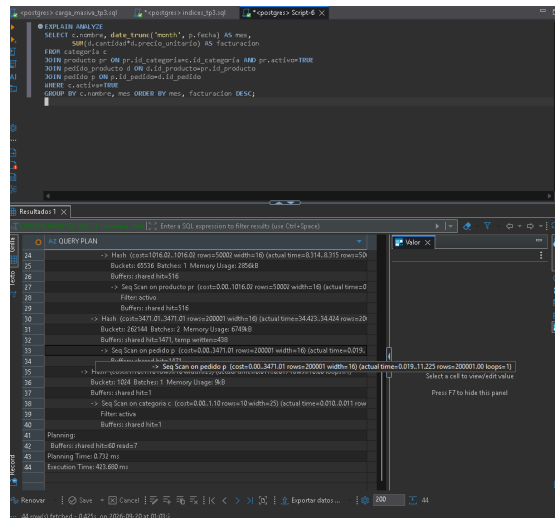
Q-B antes:



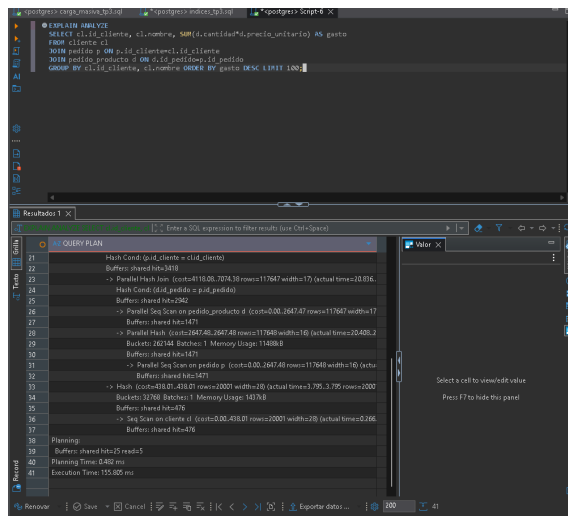
Lo que le propuse a la IA fue covering para no ir al heap: **idx_pedprod_producto_cover (id_producto,id_pedido)** **INCLUDE(cantidad,precio)** porque la PK está al revés (pedido,producto) y no sirve para el lado de producto, y **idx_pedido_cliente_cover (id_cliente,id_pedido)** porque el que tenía trae fecha/forma_pago pero no el id para el join. Los leí línea por línea, los creé en la copia, ANALYZE y volví a medir.

Después me dio **Q-A 423.680 ms** y **Q-B 155.805 ms**. O sea, prácticamente igual (una pizca mejor una, una pizca peor la otra por ruido de caché). Siguen en Hash + Seq Scan y **está bien que así sea**: con 200k filas pretender un Nested Loop serían 200k búsquedas por índice, mucho peor que hashear una vez. Lo dejo documentado como propuesta no efectiva, que el criterio lo pide igual que en TP3.

Q-A después:



Q-B después:



Consulta	Join antes	Qué puse y por qué	Join después	Mejora
Q-A facturación	3x Hash Join + external merge 430 ms	covering (producto,pedido) INCLUDE cant/precio	sigue Hash + Seq 423.68 ms	1.01x no efectiva, correcto
Q-B ranking	2x Parallel Hash 152.9 ms	covering (cliente,pedido)	sigue P-Hash 155.8 ms	0.98x ruido, no efectiva

2. Lo que la IA me explicó mal del join

Le pasé solo el texto del plan de Q-B y me lo devolvió prolijo pero con dos pifies que se repiten: me habló de **Nested Loop con tabla externa pedido_producto e interna pedido**, cuando no hay ningún Nested Loop, son **Parallel Hash Join (uno arma la hash, el otro prueba)**. Y me dijo que el **costo 4118..7074 era el tiempo total**, cuando es estimado del nodo intermedio; el total real es **Execution 155.8 ms**. También me quiso vender que con el índice nuevo pasaba a Nested Loop y mejoraba, pero el plan muestra que sigue en Hash, que es lo lógico para 200k. Lo marco en la tabla porque ahí está el 20% de lectura crítica.

3. Ranking con ventana y la de nunca vendidos

Para la Parte 3 me pedían **(a) ranking con ventana y (b) subconsulta correlacionada**, con spec precisa y equivalencia. La R1 la hice como top de gasto por cliente con **RANK() OVER (ORDER BY gasto DESC)**: si empatan comparten puesto. Me dio algo muy gráfico: **20 filas con gasto 15000 y puesto 1 en todas**. Tiene lógica porque mi carga es pareja (round-robin, cantidad 1-5 y precio 500), muchos clientes quedan empatados. La hice de dos formas, una con JOIN directo y otra con CTE + RANK, y dan lo mismo.

QUERY PLAN

```

High Cond (p.id_cliente = cl.id_cliente)
  Buffers: shared hit=3418
  Hash Right Anti Join (cost=118.00..7074.30 rows=17647 width=17) (actual time=...)
    High Cond (p.id_pedido = pid_pedido)
      Buffers: shared hit=292
      Parallel Seq Scan on pedidos_producto p (cost=0.00..2647.47 rows=17647 width=17)
        Buffers: shared hit=1471
        Parallel Hash (cost=2647.40..2647.40 rows=17647 width=10) (actual time=...)
          Buckets: 32768 Batches: 1 Memory Usage: 184368
          Buffers: shared hit=1471
          Parallel Seq Scan on pedidos p (cost=0.00..2647.40 rows=17647 width=16)
            Buffers: shared hit=1471
            Hash (cost=416.21..416.21 rows=2001 width=20) (actual time=4.621..4.622 row)
              Buckets: 32768 Batches: 1 Memory Usage: 155768
              Buffers: shared hit=475
              Seq Scan on cliente cl (cost=0.00..416.01 rows=2001 width=20) (actual time=...)
                Buffers: shared hit=476
Planning:
  Buffers: shared hit=30
Planning Time: 0.534 ms
Execution Time: 160.110 ms

```

clientes

id	nombre	gasto	punto
1	Cliente maximo 4700	15.000	1
2	Cliente maximo 3990	15.000	1
3	Cliente maximo 1800	15.000	1
4	Cliente maximo 1700	15.000	1
5	Cliente maximo 5400	15.000	1
6	Cliente maximo 1000	15.000	1
7	Cliente maximo 1520	15.000	1
8	Cliente maximo 4040	15.000	1
9	Cliente maximo 2520	15.000	1
10	Cliente maximo 2800	15.000	1
11	Cliente maximo 9030	15.000	1
12	Cliente maximo 5600	15.000	1
13	Cliente maximo 4210	15.000	1
14	Cliente maximo 1700	15.000	1
15	Cliente maximo 1100	15.000	1
16	Cliente maximo 170	15.000	1
17	Cliente maximo 570	15.000	1
18	Cliente maximo 9710	15.000	1
19	Cliente maximo 6070	15.000	1
20	Cliente maximo 1000	15.000	1

La R2 es mi favorita: **productos nunca vendidos con NOT EXISTS**. El plan es un **Hash Right Anti Join** que en **34.5 ms devuelve 2 filas: 1 Muzza y 2 Napolitana**. Cierra perfecto con la historia de la carga: los detalles apuntan solo a los 50k nuevos para no romper R1 (los viejos tienen lista 100/120 y mi precio fijo 500 los violaría), así que los 2 viejos quedan huérfanos. La versión alternativa con **LEFT JOIN ... WHERE d.id_pedido IS NULL** da las mismas 2. Verifiqué por conteo + muestra y con EXCEPT en ambos sentidos (0).

QUERY PLAN

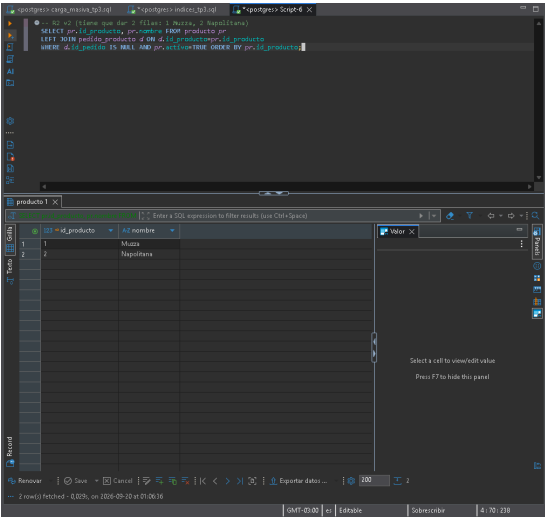
```

Hash Right Anti Join (cost=1641.00..7068.62 rows=77 width=20) (actual time=31.500..34.155 rows=2..10 loops=1)
  Hash Cond (p.id_producto = pr.id_producto)
    Buffers: shared hit=1907
    Seq Scan on pedidos_producto p (cost=0.00..3471.00 rows=10000 width=8) (actual time=0.017..6.159 ms)
      Buffers: shared hit=1471
      Hash (cost=1016.00..1016.00 rows=5000 width=20) (actual time=3.315..3.316 rows=5000.00 loops=1)
        Buckets: 85318 Batches: 1 Memory Usage: 152948
        Buffers: shared hit=516
        Seq Scan on productos pr (cost=0.00..1016.00 rows=5000 width=20) (actual time=0.012..3.095 rows=5000)
          Buffers: shared hit=516
Planning:
  Buffers: shared hit=17
Planning Time: 0.210 ms
Execution Time: 34.158 ms

```

productos

id	nombre	gasto	punto
1	Producto maximo 4700	15.000	1
2	Producto maximo 3990	15.000	1
3	Producto maximo 1800	15.000	1
4	Producto maximo 1700	15.000	1
5	Producto maximo 5400	15.000	1
6	Producto maximo 1000	15.000	1
7	Producto maximo 1520	15.000	1
8	Producto maximo 4040	15.000	1
9	Producto maximo 2520	15.000	1
10	Producto maximo 2800	15.000	1
11	Producto maximo 9030	15.000	1
12	Producto maximo 5600	15.000	1
13	Producto maximo 4210	15.000	1
14	Producto maximo 1700	15.000	1
15	Producto maximo 1100	15.000	1



4. Competencia y DUIA

Para la competencia uso la Q-A (es la analítica común con 2+ joins y agregación). Mi estrategia covering dio 430 a 423.6 (1.01x). No gana por velocidad pero sí por justificación: explico por qué Hash sigue siendo mejor que Nested y por qué el covering no alcanzó (toda la tabla igual va al hash). En la bitácora dejo las dos propuestas como no efectivas más las dos de Parte 3 aceptadas. Si en la defensa me preguntan por qué no cambió el algoritmo, digo: hash en memoria de 11 MB, Seq Scan secuencial de 1471 páginas contra 200k accesos aleatorios, y Sort a disco por agrupar 200k en 130 grupos.

Equipo	Estrategia	Antes	Después	X
Yo Q-A	covering ambos lados	430	423.6	1.01x
Yo Q-B	covering cliente	152.9	155.8	-

Uso IA	Prompt	Qué decidí
Q-A/Q-B + índices	plan real 3 Hash + spill a disco, qué nodo ataca	Los creé, no mejoraron, los documento igual
Explicar join	solo texto del plan	La corregí: no es Nested, costo no es tiempo
R1 RANK + R2 anti	spec con tablas, filtro activo, partición/orden	Acepté: 20x1 y 2 filas, EXCEPT 0